CQUniversity
AUSTRALIA

Unlocking Legal Automation with Defeasible Deontic Logic

Guido Governatori

A Legal Example: A Privacy Act

Section 1: (Prohibition to collect personal medical information)

Offence: It is an offence to collect personal medical information.
Defence: It is a defence to the prohibition of collecting personal medical information, if an entity immediately destroys the illegally collected personal medical information before making any use of the personal medical information

Section 2: An entity is permitted to collect personal medical information if the entity acts under a Court Order authorising the collection of personal medical information.

Section 3: (Prohibition to collect personal information) It is forbidden to collect personal information unless an entity is permitted to collect personal medical information.

Offence: an entity collected personal information
Defence: an entity being permitted to collect personal medical information.

CQUniversity AUSTRALIA

Computational Law

Aims of Computational Law

To provide formal methods (logic) to:

- formalise a normative system
- determine what legal requirements are in force
- determine what legal requirements have been violated or complied with

CQUniversity AUSTRALIA

Why Computational Law?

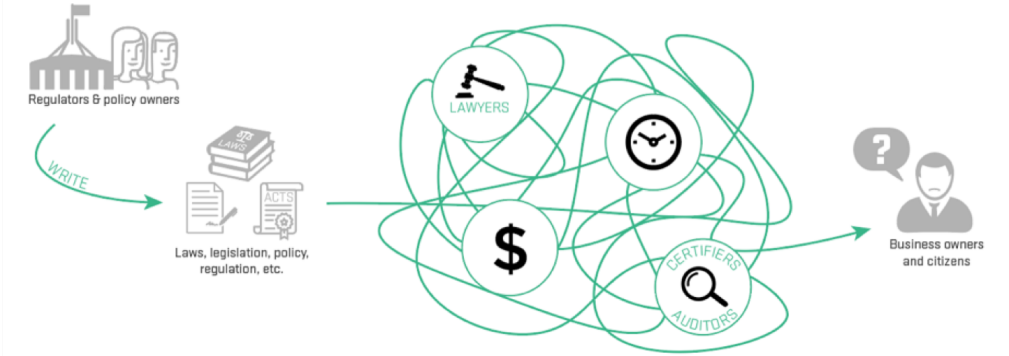
Aims

Develop and deliver technologies that identify and reduce regulatory burden and friction for government, businesses and individuals

Vision Statements

1. Enable regulators to move their rulebook from analogue to digital (machine consumable)
2. Enable government agencies to identify red tape, understand the impact of changes, provide the public with better service delivery and achieve policy agility
3. Provide 3rd parties and the regtech/lawtech industry with data and technology components so they can:
   - enable businesses and individuals to know with certainty their compliance requirements, to meet them and stay in business
   - enable public trust in businesses and trust between businesses (ethical-by-design, privacy-by-design...)

Compliance is difficult



The cost of compliance

- The cost of compliance on businesses:
  - $1.88 trillion USD per annum (Source: IRS)
  - $249 (190 in 2018) billion AUD per annum
  - in OECD countries $\sim 10 - 15\%$ GDP
- 1 million people employed in the compliance area in Australia (9% of workforce), growing fast.
- In a financial year, the Australian Government awards more than 66,000 contracts with an overall value of almost $59 billion AUD.

The cost of compliance

- The cost of compliance on businesses:
  - $1.88 trillion USD per annum (Source: IRS)
  - $249 (190 in 2018) billion AUD per annum
  - in OECD countries $\sim 10 - 15\%$ GDP
- 1 million people employed in the compliance area in Australia (9% of workforce), growing fast.
- In a financial year, the Australian Government awards more than 66,000 contracts with an overall value of almost $59 billion AUD.

Computational Law and Regulatory Technology can reduce the compliance cost by 30%

The cost of compliance

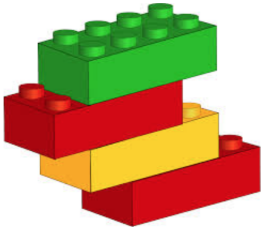
- The cost of compliance on businesses:
  - $1.88 trillion USD per annum (Source: IRS)
  - $249 (190 in 2018) billion AUD per annum
  - in OECD countries $\sim 10 - 15\%$ GDP
- 1 million people employed in the compliance area in Australia (9% of workforce), growing fast.
- In a financial year, the Australian Government awards more than 66,000 contracts with an overall value of almost $59 billion AUD.

Computational Law and Regulatory Technology can reduce the compliance cost by 30%

Automatising 5% of NSW legislation $\approx$ NSW education budget

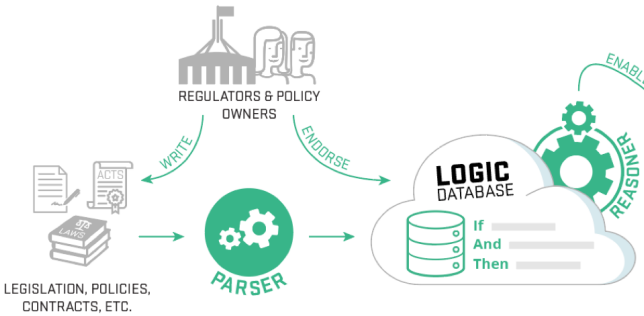
Legislation as Code

Human Understandable	• Goal • Intent
Machine Understandable	• Eligibility • Mandates • Calculations



Legislation as Code around the World

Regulation as a Platform

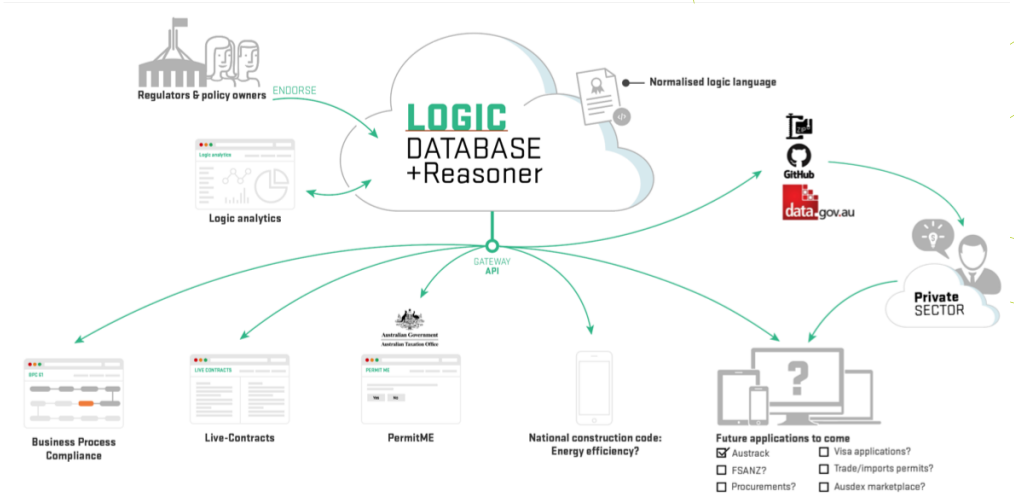


APPLICATIONS

Using the open-source logic and provided tools, anyone can build their own application, from government agencies, industry innovators and citizens.

- ☑ Websites
- ☑ Mobile apps
- ☑ Business softwares

Digital Legislation Platform



CQUniversity
AUSTRALIA

Normative Systems

Key components of Normative Systems

A normative system is a set of clauses (norms).

Key components of Normative Systems

A normative system is a set of clauses (norms).
Norms are modelled as **if** ...**then** rules

$$A_1, \ldots, A_n \Rightarrow C$$

- Definitional clauses (constitutive rules: defining terms used in a legal context)
- Normative clauses (norms defining “normative effects”)
  - obligations
  - permissions
  - prohibitions
  - violations

Key components of Normative Systems

A normative system is a set of clauses (norms).
Norms are modelled as **if** ...**then** rules

$$A_1, \ldots, A_n \Rightarrow C$$

- Definitional clauses (constitutive rules: defining terms used in a legal context)
- Normative clauses (norms defining “normative effects”)
  - obligations
  - permissions
  - prohibitions
  - violations

Norms are defeasible (handling exceptions)

Normative Effects

Obligation	A situation, an act, or a course of action to which a bearer is legally bound, and if it is not achieved or performed results in a violation.
Prohibition	A situation, an act, or a course of action which a bearer should avoid, and if it is achieved results in a violation.
Permission	Something is permitted if the prohibition or the obligation to the contrary does not hold.

Normative Effects

Obligation A situation, an act, or a course of action to which a bearer is legally bound, and if it is not achieved or performed results in a violation.

Prohibition A situation, an act, or a course of action which a bearer should avoid, and if it is achieved results in a violation.

Permission Something is permitted if the prohibition or the obligation to the contrary does not hold.

Obligations and prohibitions can be violated and violations can be compensated.

Example

Contract fragment

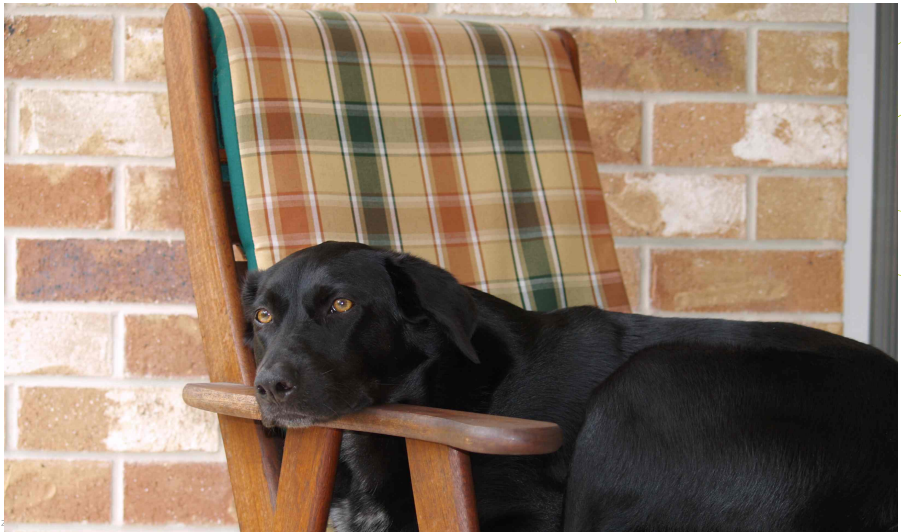
3.1 A “Premium Customer” is a customer who has spent more that $10000 in goods.

3.2 Services marked as “special order” are subject to a 5% surcharge. Premium customers are exempt from special order surcharge.

5.2 The (Supplier) shall on receipt of a purchase order for (Services) make them available within one day.

5.3 If for any reason the conditions stated in 4.1 or 4.2 are not met the (Purchaser) is entitled to charge the (Supplier) the rate of $100 for each hour the (Service) is not delivered.

Defeasibility: Reasonable results with minimum effort



Defeasibility: Reasonable results with minimum effort

Factual omniscience and (non-)monotonic reasoning

PhD → Uni

Defeasibility: Reasonable results with minimum effort

Factual omniscience and (non-)monotonic reasoning

$$PhD \rightarrow Uni$$
$$Weekend \rightarrow \neg Uni$$
$$PublicHoliday \rightarrow \neg Uni$$
$$Sick \rightarrow \neg Uni$$

Defeasibility: Reasonable results with minimum effort

Factual omniscience and (non-)monotonic reasoning

$$PhD \rightarrow Uni$$
$$Weekend \rightarrow \neg Uni$$
$$PublicHoliday \rightarrow \neg Uni$$
$$Sick \rightarrow \neg Uni$$
$$Weekend \wedge VICdeadline \rightarrow Uni$$

Defeasibility: Reasonable results with minimum effort

Factual omniscience and (non-)monotonic reasoning

$$PhD \rightarrow Uni$$
$$Weekend \rightarrow \neg Uni$$
$$PublicHoliday \rightarrow \neg Uni$$
$$Sick \rightarrow \neg Uni$$
$$Weekend \wedge VICdeadline \rightarrow Uni$$

VIC= Very Important Conference

Defeasibility: Reasonable results with minimum effort

Factual omniscience and (non-)monotonic reasoning

$$PhD \rightarrow Uni$$
$$Weekend \rightarrow \neg Uni$$
$$PublicHoliday \rightarrow \neg Uni$$
$$Sick \rightarrow \neg Uni$$
$$Weekend \wedge VICdeadline \rightarrow Uni$$
$$VICdeadline \wedge PartnerBirthday \rightarrow \neg Uni$$

Defeasibility: Reasonable results with minimum effort

Factual omniscience and (non-)monotonic reasoning

$$PhD \rightarrow Uni$$
$$Weekend \rightarrow \neg Uni$$
$$PublicHoliday \rightarrow \neg Uni$$
$$Sick \rightarrow \neg Uni$$
$$Weekend \wedge VICdeadline \rightarrow Uni$$
$$VICdeadline \wedge PartnerBirthday \rightarrow \neg Uni$$

$$Phd \wedge (\neg Weekend \vee (Weekend \wedge VICdeadline \wedge \neg PartnerBirthday)) \wedge \neg Sick \ldots \rightarrow Uni$$

Defeasiblity: Example 1

TELECOMMUNICATIONS CONSUMER PROTECTIONS CODE (C628:2012)

Section 2.1. Definitions

Complaint means an expression of dissatisfaction made to a Supplier in relation to its Telecommunications Products or the complaints handling process itself, where a response or Resolution is explicitly or implicitly expected by the Consumer.

An initial call to a provider to request a service or information or to request support is not necessarily a Complaint. An initial call to report a fault or service difficulty is not a Complaint. However, if a Customer advises that they want this initial call treated as a Complaint, the Supplier will also treat this initial call as a Complaint.

If a Supplier is uncertain, a Supplier must ask a Customer if they wish to make a Complaint and must rely on the Customer's response.

Defeasiblity: Example 2

NATIONAL CONSUMER CREDIT PROTECTION ACT 2009 (Act No. 134 of 2009) Section 29

(1) A person must not engage in a credit activity if the person does not hold a licence authorising the person to engage in the credit activity.

(3) For the purposes of subsections (1) and (2), it is a defence if:

- (a) the person engages in the credit activity on behalf of another person (the principal); and
- (b) the person is:
  - (i) an employee or director of the principal or of a related body corporate of the principal; or
  - (ii) a credit representative of the principal; and ...

CQUniversity AUSTRALIA

Defeasible Deontic Logic

Defeasible Deontic Logic Blueprint

Combination of an efficient non-monotonic logic (defeasible logic) and a deontic logic of violation.

- Derive (plausible) conclusions with the minimum amount of information.
  - Definite conclusions
  - Defeasible conclusions
- Defeasible Theory
  - Facts
  - Strict rules ($A \rightarrow B$)
  - Defeasible rules ($A \Rightarrow B$)
  - Defeaters ($A \rightsquigarrow B$)
  - Superiority relation over rules

Reasoning with Defeasible Logic

- Positive defeasible conclusions: meaning that the conclusions can be defeasible proved;
- Negative defeasible conclusions: meaning that one can show that the conclusion is not even defeasibly provable.

Proving Conclusions in Defeasible Logic

1. Give an argument for the conclusion you want to prove

Proving Conclusions in Defeasible Logic

1. Give an argument for the conclusion you want to prove
2. Consider all possible counterarguments to it

Proving Conclusions in Defeasible Logic

1. Give an argument for the conclusion you want to prove
2. Consider all possible counterarguments to it
3. Rebut all counterarguments

Proving Conclusions in Defeasible Logic

1. Give an argument for the conclusion you want to prove
2. Consider all possible counterarguments to it
3. Rebut all counterarguments
   - Defeat the argument by a stronger one
   - Undercut the argument by showing that some of the premises do not hold

Proving Conclusions in Defeasible Logic

1. Give an argument for the conclusion you want to prove
2. Consider all possible counterarguments to it
3. Rebut all counterarguments
   - Defeat the argument by a stronger one
   - Undercut the argument by showing that some of the premises do not hold

1. *A* is a fact; or

Proving Conclusions in Defeasible Logic

1. Give an argument for the conclusion you want to prove
2. Consider all possible counterarguments to it
3. Rebut all counterarguments
   - Defeat the argument by a stronger one
   - Undercut the argument by showing that some of the premises do not hold

1. A is a fact; or
2. there is an applicable rule for A, and either

Proving Conclusions in Defeasible Logic

1. Give an argument for the conclusion you want to prove
2. Consider all possible counterarguments to it
3. Rebut all counterarguments
   - Defeat the argument by a stronger one
   - Undercut the argument by showing that some of the premises do not hold

1. A is a fact; or
2. there is an applicable rule for A, and either
   1. all the rules for $\neg A$ are discarded (i.e., not applicable) or
   2. every applicable rule for $\neg A$ is weaker than an applicable rule for A.

Modelling Norms with Rules: Language

- set of propositions (boolean terms) $p, q, \ldots$

Modelling Norms with Rules: Language

- set of propositions (boolean terms) $p, q, \ldots$

Example

$customer, premiumCustomer, price(item, 100), specialOrder$

Modelling Norms with Rules: Language

- set of propositions (boolean terms) $p, q, \ldots$

Example

$customer, premiumCustomer, price(item, 100), specialOrder$

- if p is a proposition, $\sim p$ is the negation of the proposition

Modelling Norms with Rules: Language

- set of propositions (boolean terms) $p, q, \ldots$

Example

$customer, premiumCustomer, price(item, 100), specialOrder$

- if p is a proposition, $\sim p$ is the negation of the proposition

Example

$\sim premiumCustomer$ means that the customer is not a premium customer

Modelling Norms with Rules: Language

- set of propositions (boolean terms) $p, q, \ldots$

Example

$customer, premiumCustomer, price(item, 100), specialOrder$

- if p is a proposition, $\sim p$ is the negation of the proposition

Example

$\sim premiumCustomer$ means that the customer is not a premium customer

,

- if p is a proposition, then $[O]p$ means that p is obligatory, $[P]p$ means that p is permitted

Modelling Norms with Rules: Language

- set of propositions (boolean terms) $p, q, \ldots$

Example

$customer$, $premiumCustomer$, $price(item, 100)$, $specialOrder$

- if p is a proposition, $\sim p$ is the negation of the proposition

Example

$\sim premiumCustomer$ means that the customer is not a premium customer

,

- if p is a proposition, then $[O]p$ means that p is obligatory, $[P]p$ means that p is permitted

Example

$[O]payInvoice$ means that it is obligatory to pay the invoice, $[P]chargeSurcharge$ means that it is permitted to charge the surcharge

Modelling Norms with Rules: Rules

- a constitutive rule

$$a_1, \ldots, a_n \Rightarrow p$$

the conclusion of the rule is not in the scope of an obligation or permission

$$customer, moreThan10000\$ \Rightarrow premiumCustomer$$

Modelling Norms with Rules: Rules

- a constitutive rule

$$a_1, \ldots, a_n \Rightarrow p$$

the conclusion of the rule is not in the scope of an obligation or permission

$$customer, moreThan10000\$ \Rightarrow premiumCustomer$$

- a rule

$$a_1, \ldots, a_n \Rightarrow [O]p$$

the conclusion of the rule is in the scope of an obligation

$$supplier, purchaseOrder \Rightarrow [O]deliverWithinOneDay$$

Modelling Exceptions

- Rules can be defeasible (i.e., they can be overridden by other rules)

Modelling Exceptions

- Rules can be defeasible (i.e., they can be overridden by other rules)
- A rule r_1 is overridden by a rule r_2 if the conclusion of r_1 is the negation of the conclusion of r_2 and the premises of r_2 are satisfied

$$r_1 : specialOrder \Rightarrow chargeSurcharge$$
$$r_2 : premiumCustomer \Rightarrow \sim chargeSurcharge$$
$$r_2 > r_1$$

Modelling Violations

Definition

Violation A violation means that an obligation or a prohibition has not been fulfilled. A violation can be compensated by another obligation.

Modelling Violations

Definition

Violation A violation means that an obligation or a prohibition has not been fulfilled. A violation can be compensated by another obligation.

$$[O]p \quad \sim p$$
$$[O]\sim p \quad p$$

Modelling Violations

Definition

Violation A violation means that an obligation or a prohibition has not been fulfilled. A violation can be compensated by another obligation.

$$[O]p \quad \sim p$$
$$[O]\sim p \quad p$$

$$[O]payInvoice \quad \sim payInvoice$$
$$[O]\sim smoking \quad smoking$$

Modelling Violations: Compensation (or penalties)

Definition

Compensation A compensation is an obligation that is activated when a violation occurs.

Modelling Violations: Compensation (or penalties)

Definition

Compensation A compensation is an obligation that is activated when a violation occurs.

$$\cdots \Rightarrow [O]p \otimes [O]q$$

Modelling Violations: Compensation (or penalties)

Definition

Compensation A compensation is an obligation that is activated when a violation occurs.

$$\cdots \Rightarrow [O]p \otimes [O]q$$

Example: if it is obligatory to pay the invoice and the invoice is not paid, then it is obligatory to pay a fines

$$invoiceSent \Rightarrow [O]payInvoice \otimes [O]payFine$$

Compensable vs Non-compensable

- *compensable obligations*: fulfilling the penalty related to the violation of the obligation makes the process compliant.
  ‘Each complaint must be either resolved or escalated and reported to the Telecommunication Ombudsmen’
- *non-compensable obligations*: violating the obligation makes the process non-compliant.
  ‘To pass the course a student has to pass the final exam’

Compensations or Acceptable Alternatives?

TCPC 2012, Section 8.1.1

...implement, operate and comply with a Complaint handling process that:

(vii) requires all Complaints to be:

A. Resolved in an objective, efficient and fair manner; and
B. escalated and managed under the Supplier's internal escalation process if requested by the Consumer or a former Customer.

Compensations or Acceptable Alternatives?

TCPC 2012, Section 8.1.1

...implement, operate and comply with a Complaint handling process that:

(vii) requires all Complaints to be:

A. Resolved in an objective, efficient and fair manner; and
B. escalated and managed under the Supplier's internal escalation process if requested by the Consumer or a former Customer.

YAWL Deed of Assignment, Clause 5.2.

Each Contributor indemnifies and will defend the Foundation against any claim, liability, loss, damages, cost and expenses suffered or incurred by the Foundation as a result of any breach of the warranties given by the Contributor under **clause 5.1**.

CQUniversity
AUSTRALIA

Modelling Norms in Defeasible Deontic Logic

Legal Coding

- Legal coding is the process of translating legal rules and regulations into a machine-readable format.
- The goal of legal coding is to enable automated compliance checking and enforcement, as well as to facilitate the development of regulatory technology (RegTech) solutions.

Legal Coding: Challenges

- Legal coding is a form of legal interpretation.
- A legal coder can encode 5/6 pages of legal text per day.
- Different coders may interpret the same legal text differently, leading to different legal codes.
- Different coders might translate the same interpretation in different ways.
- Legal coding should be maintainable (legal rules change, and the legal code should be updated accordingly).
- Legal coding should be verifiable (the legal code should be verified against the original legal text to ensure that it is correct).
- Legal coding should be explainable (the legal code should be explainable to non-technical stakeholders, such as lawyers and regulators).
- It should be possible to code in team.

Legal Coding Methodology

- Legal coding should be based on a clear methodology that guides the coder through the process of translating legal text into code.
- The methodology should include steps for:
  - Analysing the legal text to identify the relevant legal rules and concepts.
  - Designing a formal representation of the legal rules and concepts.
  - Implementing the formal representation in a machine-readable format.
  - Verifying the legal code against the original legal text.
  - Maintaining the legal code over time as legal rules change.

Encoding Assumptions

A normative system is a set of clauses (norms).

Encoding Assumptions

A normative system is a set of clauses (norms).
Norms are modelled as **if** ...**then** rules

$$A_1, \ldots, A_n \Rightarrow C$$

- Definitional clauses (constitutive rules: defining terms used in a legal context)
- Prescriptive clauses (norms defining “normative effects”)
  - obligations
  - permissions
  - prohibitions
  - violations

Encoding Assumptions

A normative system is a set of clauses (norms).
Norms are modelled as **if** ...**then** rules

$$A_1, \ldots, A_n \Rightarrow C$$

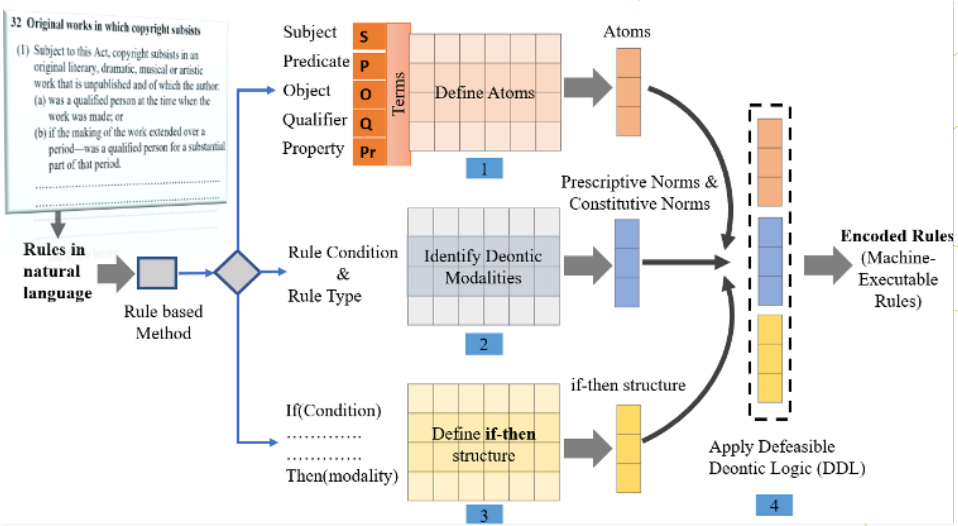
- Definitional clauses (constitutive rules: defining terms used in a legal context)
- Prescriptive clauses (norms defining “normative effects”)
  - obligations
  - permissions
  - prohibitions
  - violations

Norms are defeasible (handling exceptions)

Encoding Pipeline

1. Read the entire document to understand its meaning, intent and purpose
2. Establish the encoding convention (e.g., naming convention for atoms, rules, etc.)
3. Identify the Atoms, and the relations among the terms.
4. Identify the definitions of the terms and concepts
5. Identify the deontic modalities (obligations, permissions, and prohibitions)
6. Identify the structure of the rules and the relationships between the rules
7. Identify the relations between the rules

Encoding Methodology



Identify Atoms

An Atom (or atomic proposition) is a basic statement that can be either true or false.

- Look for nouns and noun phrases
- Identify the subject and object of sentences
- Identify the main verb or action in the sentence
- Look for adjectives and adverbs that modify the nouns or verbs
- Identify relationships between entities

Modelling Atoms

An atom is an atomic Boolean proposition

- Subject + Property

Modelling Atoms

An atom is an atomic Boolean proposition

- Subject + Property
- Subject + Predicate + Object

Modelling Atoms

An atom is an atomic Boolean proposition

- Subject + Property
- Subject + Predicate + Object
- Subject + Predicate + Object + Object

Modelling Atoms

An atom is an atomic Boolean proposition

- Subject + Property
- Subject + Predicate + Object
- Subject + Predicate + Object + Object
- Subject (+ Qualifier) + Predicate (+ Qualifier) + Object (+ Qualifier)

Most Common Patterns

- S-P-O: Subject-Predicate-Object;
- S-P-Q-O: Subject-Predicate-Qualifier-Object;
- S-Pr: Subject-Property,
- S-P-O-O: Subject-Predicate-Object-Object;
- S-Q-P-O: Subject-Qualifier-Predicate-Object.

Subject-Predicate-Object

the copyright subsists in a work

Subject-Predicate-Object

the copyright subsists in a work

- Subject: `copyright`

Subject-Predicate-Object

the copyright subsists in a work

- Subject: `copyright`
- Predicate: `subsist in`

Subject-Predicate-Object

the copyright subsists in a work

- Subject: `copyright`
- Predicate: `subsist in`
- Object: `work`

Subject-Predicate-Object

the copyright subsists in a work

- Subject: `copyright`
- Predicate: `subsist in`
- Object: `work`

Subject-Predicate-Object

the copyright subsists in a work

- Subject: `copyright`
- Predicate: `subsist in`
- Object: `work`

$$predicate(subject, object)$$
$$subjectPredicateObject$$

hence, for the specific sentence we have

$$subsistIn(copyright, work)$$
$$copyrightSubsistInWork$$

Subject-Predicate-Qualifier-Object

the person reproduces the work in material form

- Subject: `person`

Subject-Predicate-Qualifier-Object

the person reproduces the work in material form

- Subject: `person`
- Predicate: `reproduce`

Subject-Predicate-Qualifier-Object

the person reproduces the work in material form

- Subject: `person`
- Predicate: `reproduce`
- Qualifier: `in material form`

Subject-Predicate-Qualifier-Object

the person reproduces the work in material form

- Subject: `person`
- Predicate: `reproduce`
- Qualifier: `in material form`
- Object: `work`

Subject-Predicate-Qualifier-Object

the person reproduces the work in material form

- Subject: `person`
- Predicate: `reproduce`
- Qualifier: `in material form`
- Object: `work`

Subject-Predicate-Qualifier-Object

the person reproduces the work in material form

- Subject: `person`
- Predicate: `reproduce`
- Qualifier: `in material form`
- Object: `work`

$$predicate(subject, qualifier, object)$$
$$predicateQualifier(subject, object)$$
$$subjectPredicateQualifierObject$$

thus, we can model it as either

$$reproduce(person, inMaterialForm, work)$$
$$reproduceInMaterialForm(person, work)$$
$$personReproduceInMaterialFormWork$$

Subject-Property

the person is a premium customer

- Subject: `person`

Subject-Property

the person is a premium customer

- Subject: `person`
- Property: `premiumCustomer`

Subject-Property

the person is a premium customer

- Subject: `person`
- Property: `premiumCustomer`

Subject-Property

the person is a premium customer

- Subject: `person`
- Property: `premiumCustomer`

property(*subject*)

subjectProperty

Subject-Property

the person is a premium customer

- Subject: `person`
- Property: `premiumCustomer`

property(*subject*)
subjectProperty

hence, we can model it as either

premiumCustomer(*person*)
personPremiumCustomer

Subject-Property (alternative)

a premium customer is a customer that…
work means literary […] work

- Subject: `customer` / `work`

Subject-Property (alternative)

a premium customer is a customer that…
work means literary […] work

- Subject: `customer` / `work`
- Property: `premiumCustomer` / `literaryWork`

Subject-Property (alternative)

a premium customer is a customer that…
work means literary […] work

- Subject: `customer` / `work`
- Property: `premiumCustomer` / `literaryWork`

Subject-Property (alternative)

a premium customer is a customer that…
work means literary […] work

- Subject: `customer` / `work`
- Property: `premiumCustomer` / `literaryWork`

hence, we can model it as either

premiumCustomer
literaryWork

Subject-Property-Object-Object

a person has authorised the reproduction of a work

- Subject: `person`

Subject-Property-Object-Object

a person has authorised the reproduction of a work

- Subject: `person`
- Predicate: `authorise`

Subject-Property-Object-Object

a person has authorised the reproduction of a work

- Subject: `person`
- Predicate: `authorise`
- Object: `reproduction`

Subject-Property-Object-Object

a person has authorised the reproduction of a work

- Subject: `person`
- Predicate: `authorise`
- Object: `reproduction`
- Object: `work`

Subject-Property-Object-Object

a person has authorised the reproduction of a work

- Subject: `person`
- Predicate: `authorise`
- Object: `reproduction`
- Object: `work`

Subject-Property-Object-Object

a person has authorised the reproduction of a work

- Subject: `person`
- Predicate: `authorise`
- Object: `reproduction`
- Object: `work`

predicate(*subject*, *object1*, *object2*)

subjectPredicateObject1Object2

Subject-Property-Object-Object

a person has authorised the reproduction of a work

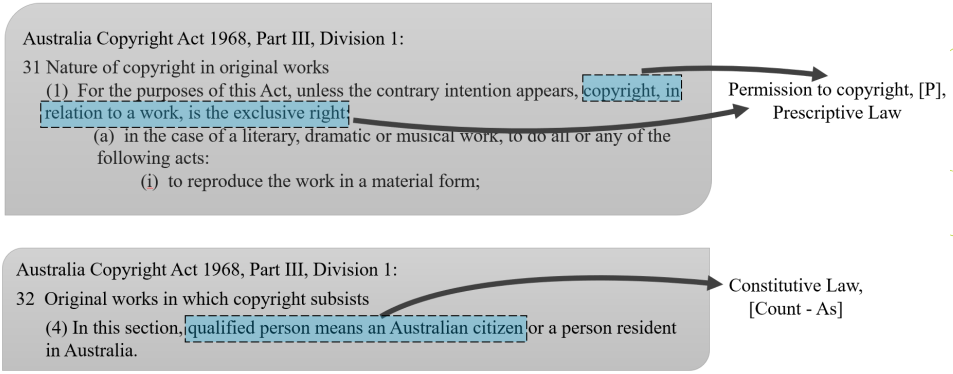
- Subject: `person`
- Predicate: `authorise`
- Object: `reproduction`
- Object: `work`

predicate(*subject*, *object1*, *object2*)
subjectPredicateObject1Object2

hence, we can model it as either

authorise(*person*, *reproduction*, *work*)
personAuthoriseReproductionWork

Identifying Deontic Modalities



Identifying Rules

- Look for conditional statements (if-then, when, whenever, etc.)
- Identify the antecedent (the “if” part) and the consequent (“then” part)
- Look for logical connectors (and, or, not) within the antecedent and consequent
- Identify any exceptions or special cases mentioned in the text
- Determine the type of rule (constitutive or regulative (prescriptive or permissive.))

Example of a Rule

31. Nature of copyright in original works

(1) For the purposes of this Act, unless the contrary intention appears, copyright, in relation to a work, is the exclusive right:

(a) in the case of a literary, dramatic or musical work, to do all or any of the following acts:

(i) to reproduce the work in a material form;

The Structure of the Rule

```
r_{31.1.a.i}
IF
   AND
      copyrightSubsistInWork
      personOwnCopyright
      OR
        literaryWork
        dramaticWork
        musicalWork
THEN [P]
    personReproduceMaterialFormWork
```

The Structure of the Rule 2

```
r_{31}
IF
  AND
    copyrightSubsistInWork
    NEG personOwnCopyright
    OR
        literaryWork
        dramaticWork
        musicalWork
THEN [O]
    NEG personReproduceMaterialFormWork
--
r_{31.1}
IF
    oppositeIntentionAppearCopyright
THEN (definition)
    NEG copyrightSubsistInWork
```

Modelling ORs

The coding language does not support the direct use of ORs:

```
a AND b AND (c OR d OR e) => f
```

Modelling ORs

The coding language does not support the direct use of ORs:

```
a AND b AND (c OR d OR e) => f
```

OPTION 1: create multiple rules for each combination of the ORs:

```
a AND b AND c => f
a AND b AND d => f
a AND b AND e => f
```

Modelling ORs

The coding language does not support the direct use of ORs:

```
a AND b AND (c OR d OR e) => f
```

OPTION 1: create multiple rules for each combination of the ORs:

```
a AND b AND c => f
a AND b AND d => f
a AND b AND e => f
```

OPTION 2: create a new atom for the ORs:

```
c => newAtom
d => newAtom
e => newAtom
a AND b AND newAtom => f
```

CQUniversity AUSTRALIA

From Theory to Practice

DDL in ASP

We use the implementation of Defeasible Deontic Logic in Answer Set Programming

`https://github.com/gvdgdo/Defeasible-Deontic-Logic`

A few alternative to model rules in DDL-ASP

Tutorial Material

https://github.com/gvdgdo/uladdl

DDL in ASP: Option 1

$$label : a_1, \ldots, a_n, [O](o_1), \ldots [O](o_m), [P](p_1), \ldots [P](p_k) \Rightarrow_X head$$

```
<type>Rule(label, head).
body(label, a_1).
...
body(label,obl(o_1)).
...
body(label,per(p_1)).
...
```

DDL in ASP: Option 1 head

$$head = c_1 \otimes c_2 \otimes \cdots \otimes c_n$$

```
compensate(label, c_n, c_n+1, n).
```

DDL in ASP: Option 2

$$label : a_1, \ldots, a_n, [O](o_1), \ldots [O](o_m), [P](p_1), \ldots [P](p_k) \Rightarrow_X head$$

```
label: a_1 & ... & a_n & [O]o_1 & ... & [O]o_m
       & [P]p_1 & ... & [P]p_k => [O]c_1 & ... & [O]c_n
```

Do not split the rule in multiple lines!

And use the provided parser to translate the rules into the internal format.

A negated literal/atom is written as `~atom` and represented in DDL-ASP as `non(atom)`.

DDL in ASP

- every atom must be declared as `atom(name)` (when using the Turnip syntax the atoms are not declared but inferred from the rules)
- the superiority relation is written as > or < and the implementation has `superior(label1,label1)` and `inferior(label1,label2)`

A Privacy Act

Section 1: (Prohibition to collect personal medical information)

Offence: It is an offence to collect personal medical information.

Defence: It is a defence to the prohibition of collecting personal medical information, if an entity immediately destroys the illegally collected personal medical information before making any use of the personal medical information

Section 2: An entity is permitted to collect personal medical information if the entity acts under a Court Order authorising the collection of personal medical information.

Section 3: (Prohibition to collect personal information) It is forbidden to collect personal information unless an entity is permitted to collect personal medical information.

Offence: an entity collected personal information

Defence: an entity being permitted to collect personal medical information.

Making Sense of the Act

- Collection of medical information is forbidden.
- Destruction of the illegally collected medical information excuses the illegal collection.
- Collection of medical information is permitted if there is an authorising court order.
- Collection of personal information is forbidden.
- Collection of personal information is permitted if the collection of medical information is permitted

Logical Structure of the Act

- *b* (“collection of medical information”) is forbidden
  - *c* (“destruction of medical information”) compensates the illegal collection
- *b* is permitted if *a* (“acting under a court order”)
- *d* (“collection of personal information”) is forbidden
- *d* is permitted if *b* is permitted

Encoding the Act

```
r1: => [O]~entityCollectMedicalInfo & [O]entityImmediatelyDestroyMedicalInfo
r2: entityAuthorisedCollectionCourtOrder => [P]entityCollectMedicalInfo
r3: => [O]~entityCollectPersonalInfo
r4: [P]entityCollectMedicalInfo => [P]entityCollectPersonalInfo
r2 > r1
r4 > r3
```

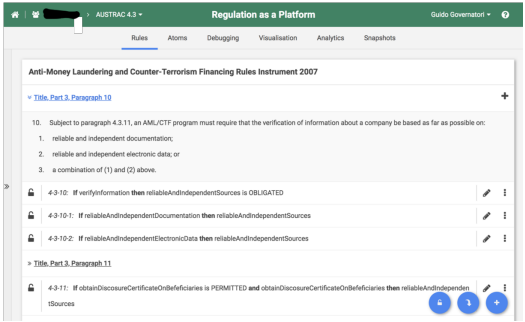
CQUniversity AUSTRALIA

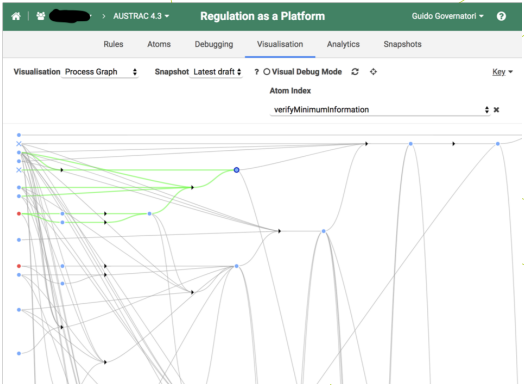
Computational Law in the Wild

Applications

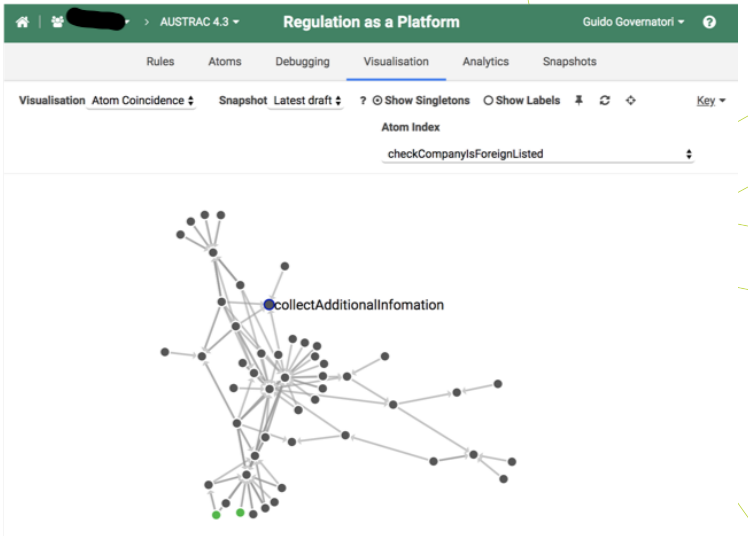
- Regulation as a Platform (RaaP)
- Regorous: Business Process Compliance
- License Composition
- Business Process Compliance for EU AI Act
- Autonomous Vehicles
- Question Answering

Regulation as a Platform

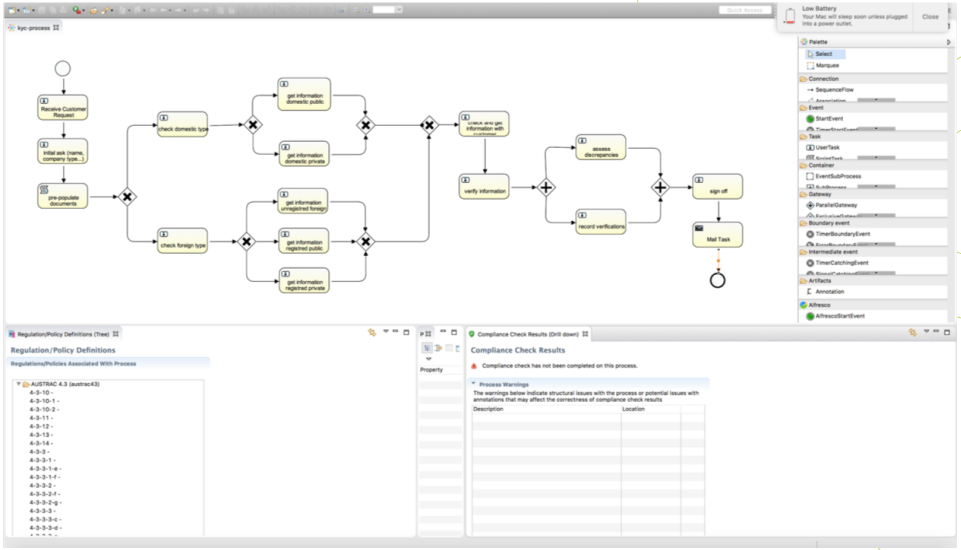




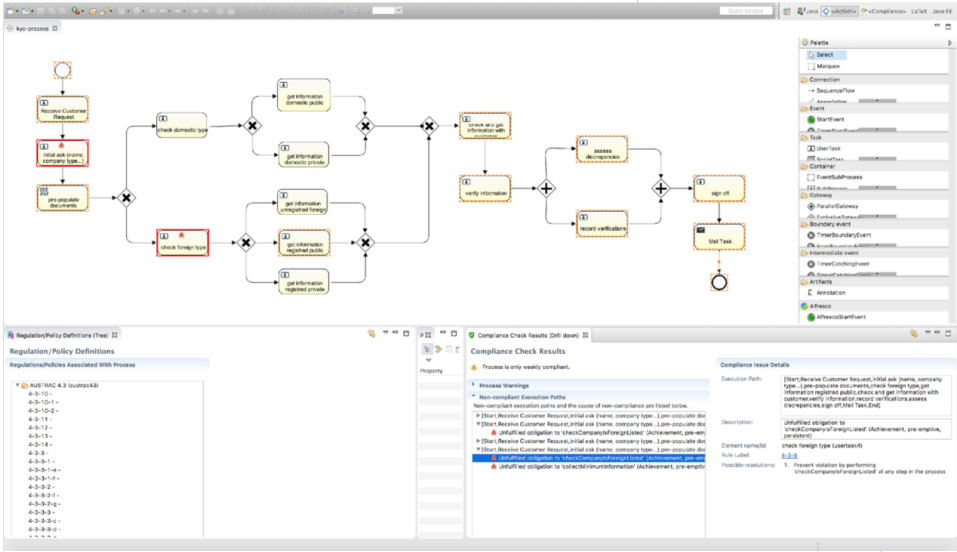
Regulation as a Platform



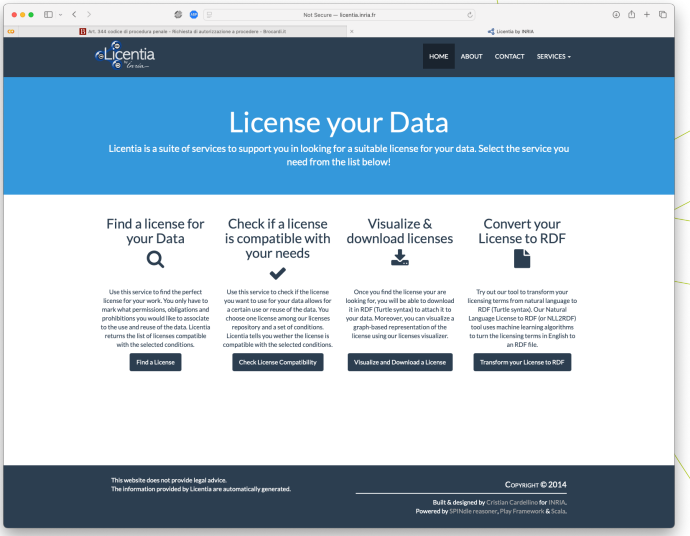
Business Process Compliance



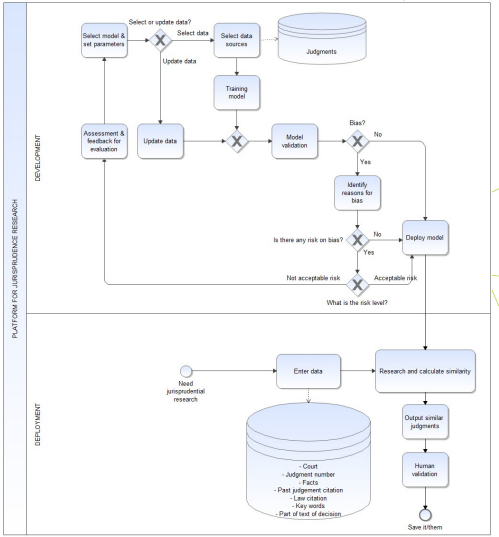
Business Process Compliance



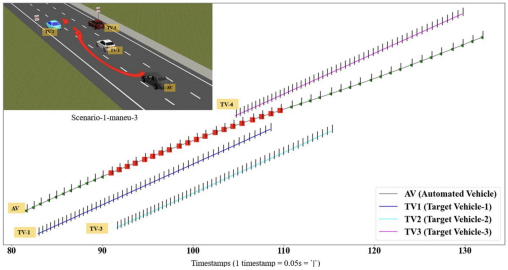
License Composition

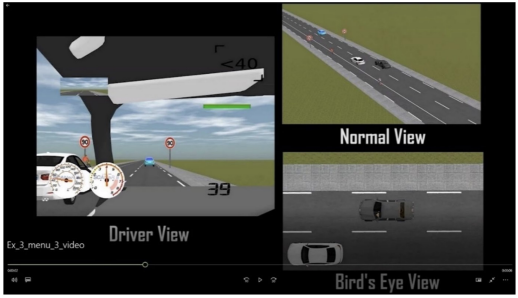


BPC for EU AI Act



Autonomous Vehicles





Autonomous Vehicles: Validation

- encoded Queensland Road Rules (sections on Overtaking)
- generated a set of test scenarios
- for each scenarios we generated a set of test cases (clear-cases, borderline cases)
- executed the test cases on a simulator
- asked human drivers (expert and regular drivers) to evaluate the test cases
- compared the evaluation by human drivers with the automated evaluation
  - 85/90% of agreement for clear cases
  - 55% of agreement for borderline cases

Question Answering

- Australian (Food Outlet) Licensing conditions
- Doctor Roster Automated Compliance Checking
- Doctor Roster Automated Scheduling (according to legal requirements)
- Decision tree for EU AI Act